

Sudoku Solver Pseudocode

Minigrid Permutations, Compatibility Graphs, and Exact Support Search

1 Notation

The solver works with an $N \times N$ Sudoku board and $K \times K$ minigrids. For the standard puzzle, $N = 9$ and $K = 3$. The implementation treats each minigrid completion as a graph vertex. A full solution is an assignment of one compatible permutation to each minigrid.

- B is the input board. A zero cell is empty.
- M stores row, column, box, and per-cell conflict masks.
- P_i is the set of generated permutations for minigrid i .
- G is the compatibility graph over all minigrid permutations.
- D_i is the current search domain of permutation ids for minigrid i .
- A_i is the selected permutation id for minigrid i , if assigned.

2 Complete Solver Pipeline

This algorithm corresponds to `SudokuSolver::solve_with_stats` in `src/solver/mod.rs`. It is the top-level pipeline used by the full reporting path.

Listing 1: Complete solver pipeline

```
procedure SolveWithStats(board B)
  start total timer

  // Phase 1: fixed conflicts from the input board
  M <- GenerateMasks(B)

  // Phase 1.5: deterministic preprocessing
  H <- copy(B)
  filled <- ApplyHiddenSingles(H, M)

  // Phase 2: local minigrid search
  for each minigrid id i in 0 .. N - 1 do in parallel
    P_i <- GenerateMinigridPermutations(H, M, i)
  end for

  // Phase 3: graph of compatible local completions
  G <- NewGraph(P_0, P_1, ..., P_{N-1})
  BuildCompatibilityEdges(G)

  // Phase 4: exact support pruning
  (removed, configurations) <- ExactSupportPrune(G)
```

```

// Phase 5: reconstruct boards from full configurations
solutions <- ExtractSolutions(G, configurations)
class <- Classify(solutions)

return SolveReport(solutions, statistics, class)
end procedure

```

3 Mask Initialization

This algorithm corresponds to `Masks::generate` in `src/types/masks.rs`. The masks reject invalid givens and provide constant-time conflict checks for later stages.

Listing 2: Generate row, column, box, and conflict masks

```

procedure GenerateMasks(board B)
  initialize rows[0 .. N-1] as empty masks
  initialize cols[0 .. N-1] as empty masks
  initialize boxes[0 .. N-1] as empty masks
  initialize conflict[0 .. N-1][0 .. N-1] as empty masks

  for r in 0 .. N - 1 do
    for c in 0 .. N - 1 do
      d <- B[r][c]
      if d = 0 then
        continue
      end if

      b <- BoxIndex(r, c)
      if rows[r] contains d or cols[c] contains d or boxes[b] contains d then
        error "invalid board: duplicate given"
      end if

      add d to rows[r]
      add d to cols[c]
      add d to boxes[b]
    end for
  end for

  for r in 0 .. N - 1 do
    for c in 0 .. N - 1 do
      b <- BoxIndex(r, c)
      conflict[r][c] <- rows[r] union cols[c] union boxes[b]
    end for
  end for

  return Masks(rows, cols, boxes, conflict)
end procedure

```

4 Hidden Single Deduction

This algorithm corresponds to `apply_hidden_singles` in `src/solver/heuristics.rs`. It fills digits that have one legal position in a row, column, or minigrid.

Listing 3: Hidden single deduction

```

procedure ApplyHiddenSingles(board B, masks M)

```

```

total_filled <- 0

repeat
  changed <- false

  for d in 1 .. N do
    for each row r do
      if FillIfOnlyLegalCell(B, M, d, cells in row r) then
        total_filled <- total_filled + 1
        changed <- true
      end if
    end for

    for each column c do
      if FillIfOnlyLegalCell(B, M, d, cells in column c) then
        total_filled <- total_filled + 1
        changed <- true
      end if
    end for

    for each box b do
      if FillIfOnlyLegalCell(B, M, d, cells in box b) then
        total_filled <- total_filled + 1
        changed <- true
      end if
    end for
  end for
until changed = false

return total_filled
end procedure

function FillIfOnlyLegalCell(board B, masks M, digit d, cell_set S)
  count <- 0
  possible <- None

  for each cell (r, c) in S do
    if B[r][c] = 0 and d not in M.conflict[r][c] then
      possible <- Some(r, c)
      count <- count + 1
    end if
  end for

  if count = 1 then
    (r, c) <- possible
    B[r][c] <- d
    UpdateMasksAfterPlacement(M, r, c, d)
    return true
  end if

  return false
end function

```

Listing 4: Update masks after a deterministic placement

```

procedure UpdateMasksAfterPlacement(masks M, row r, column c, digit d)
  b <- BoxIndex(r, c)
  add d to M.rows[r]

```

```

add d to M.cols[c]
add d to M.bboxes[b]

for each column x in 0 .. N - 1 do
  bx <- BoxIndex(r, x)
  M.conflict[r][x] <- M.rows[r] union M.cols[x] union M.bboxes[bx]
end for

for each row y in 0 .. N - 1 do
  by <- BoxIndex(y, c)
  M.conflict[y][c] <- M.rows[y] union M.cols[c] union M.bboxes[by]
end for

for each cell (y, x) inside box b do
  bx <- BoxIndex(y, x)
  M.conflict[y][x] <- M.rows[y] union M.cols[x] union M.bboxes[bx]
end for
end procedure

```

5 Minigrid Permutation Generation

This algorithm corresponds to `generate_all_permutations` and `PermutationGenerator::dfs` in `src/solver/permutations.rs`. Each minigrid is solved locally. The search respects fixed row, column, and box conflicts from the current board and masks, but it does not yet enforce compatibility with other minigrids beyond the fixed cells already present in the heuristic board.

Listing 5: Generate all minigrid permutations

```

procedure GenerateAllMinigridPermutations(board B, masks M)
  for each minigrid id i in 0 .. N - 1 do in parallel
    minigrid <- ExtractMinigrid(B, i)
    generated[i] <- GenerateMinigridPermutations(minigrid, M)
  end for

  return generated
end procedure

procedure GenerateMinigridPermutations(minigrid g, masks M)
  used <- M.bboxes[g.id]
  nodes <- empty list
  payloads <- empty list

  DFSMinigrid(g, M, used, nodes, payloads)

  return GeneratedMinigrid(nodes, payloads)
end procedure

```

Listing 6: Depth-first minigrid completion with MRV

```

procedure DFSMinigrid(minigrid g, masks M, mask used, nodes, payloads)
  choice <- FindBestCell(g, M, used)

  if choice = Some(index, conflict) then
    candidates <- digits not present in conflict

    for each digit d in candidates do
      g.cells[index] <- d
    end for
  end if
end procedure

```

```

        remove index from g.empty

        next_used <- used union {d}
        DFSMinigrid(g, M, next_used, nodes, payloads)

        g.cells[index] <- 0
        add index to g.empty
    end for

    else if used contains all digits 1 .. N then
        payload_id <- length(payloads)
        append copy(g.cells) to payloads
        append PermutationNode(g.cells, payload_id) to nodes

    else
        return // dead end
    end if
end procedure

```

Listing 7: Select the most constrained empty cell inside a minigrid

```

function FindBestCell(minigrid g, masks M, mask used)
    best <- None
    best_forbidden_count <- 0

    for each local index i in g.empty do
        (r, c) <- GlobalCoordinates(g.id, i)
        conflict <- M.conflict[r][c] union used

        if conflict contains all digits 1 .. N then
            return None
        end if

        forbidden_count <- CountDigits(conflict)
        if forbidden_count > best_forbidden_count then
            best <- Some(i, conflict)
            best_forbidden_count <- forbidden_count

            if forbidden_count = N - 1 then
                break
            end if
        end if
    end for

    return best
end function

```

6 Compatibility Graph Construction

This algorithm corresponds to `Graph::new`, `Graph::create_edges`, and `PermutationNode` row and column compatibility in `src/types/graph`. The graph connects compatible permutations from related minigrids. Unrelated minigrids do not receive adjacency tables.

Listing 8: Build the graph container and signature indexes

```

procedure NewGraph(generated minigrids P)
    for each minigrid id i do

```

```

graph.minigrids[i] <- P_i.nodes
graph.payloads[i] <- P_i.payloads
graph.row_index[i] <- BuildAxisIndex(P_i.nodes, use_rows = true)
graph.col_index[i] <- BuildAxisIndex(P_i.nodes, use_rows = false)
end for

graph.pair_tables <- empty list
graph.pair_lookup <- empty matrix
return graph
end procedure

procedure BuildAxisIndex(nodes, use_rows)
  signatures <- empty list
  signature_to_id <- empty map
  perm_to_signature <- empty list
  signature_to_perms <- empty bitset list

  for each permutation id p and node n in nodes do
    if use_rows then
      signature <- row masks of n
    else
      signature <- column masks of n
    end if

    if signature is not in signature_to_id then
      assign next signature id
      append empty bitset to signature_to_perms
    end if

    s <- signature_to_id[signature]
    perm_to_signature[p] <- s
    add p to signature_to_perms[s]
  end for

  return AxisIndex(signatures, perm_to_signature, signature_to_perms)
end procedure

```

Listing 9: Create compatibility edges between related minigrids

```

procedure BuildCompatibilityEdges(graph G)
  clear G.pair_tables

  for left in 0 .. N - 1 do
    for right in left + 1 .. N - 1 do
      relation <- RelationshipBetween(left, right)
      if relation = NotRelated then
        continue
      end if

      if relation = RowRelated then
        L <- G.row_index[left]
        R <- G.row_index[right]
      else
        L <- G.col_index[left]
        R <- G.col_index[right]
      end if

      left_to_right <- empty table indexed by signatures of L
    end for
  end for

```

```

        right_to_left <- empty table indexed by signatures of R

        for each signature id a in L do
            for each signature id b in R do
                if SignaturesCompatible(L.signatures[a], R.signatures[b]) then
                    left_to_right[a] <- left_to_right[a] union R.
signature_to_perms[b]
                    right_to_left[b] <- right_to_left[b] union L.
signature_to_perms[a]
                end if
            end for
        end for

        append PairAdj(left, right, relation, left_to_right, right_to_left) to G.
pair_tables
    end for
end for

G.pair_lookup <- BuildPairLookup(G.pair_tables)
end procedure

```

Listing 10: Relationship and signature compatibility

```

function RelationshipBetween(a, b)
    same_block_row <- floor(a / K) = floor(b / K)
    same_block_col <- (a mod K) = (b mod K)

    if same_block_row and same_block_col then
        return NotRelated
    else if same_block_row then
        return RowRelated
    else if same_block_col then
        return ColRelated
    else
        return NotRelated
    end if
end function

function SignaturesCompatible(signature A, signature B)
    for local_axis in 0 .. K - 1 do
        if A[local_axis] intersects B[local_axis] then
            return false
        end if
    end for

    return true
end function

```

7 Local Support Pruning

This algorithm corresponds to `Pruner::run_local` in `src/solver/pruning.rs`. It removes permutations that have no currently live compatible neighbor in at least one related minigrid. This is a local consistency pass.

Listing 11: Iterative local support pruning

```

procedure LocalSupportPrune(graph G)

```

```

for each minigrid i do
  alive[i] <- bitset containing every permutation id in G[i]
end for

repeat
  changed <- false

  for each minigrid i do
    for each permutation p in G[i] do
      if p is not alive[i] then
        continue
      end if

      supported <- true

      for each minigrid j do
        if RelationshipBetween(i, j) = NotRelated then
          continue
        end if

        compatible <- CompatibleSet(G, i, p, j)
        if compatible contains no permutation q with alive[j][q] = true
      then
        supported <- false
        break
      end if
    end for

    if supported = false then
      remove p from alive[i]
      changed <- true
    end if
  end for
end for
until changed = false

keep <- alive permutation ids for each minigrid
RetainPermutations(G, keep)
return number of removed permutations
end procedure

```

8 Exact Support Pruning

This algorithm corresponds to `Pruner::run` in `src/solver/pruning.rs`. It first applies local pruning, then runs the exact configuration search. A permutation survives only if some full valid configuration uses it.

Listing 12: Exact global support pruning

```

procedure ExactSupportPrune(graph G)
  pre_local_counts <- permutation counts of G

  LocalSupportPrune(G)

  current_counts <- permutation counts of compacted G
  configurations <- SearchAllConfigurations(G)

```



```

for each minigrid i do
    supported[i] <- array of false values sized by current_counts[i]
end for

for each configuration C in configurations do
    for each minigrid i do
        p <- C[i] // p is an id in the post-local-prune graph
        supported[i][p] <- true
    end for
end for

for each minigrid i do
    keep[i] <- all current permutation ids p where supported[i][p] = true
    remap[i] <- map current kept ids to new compact ids
end for

removed <- total_pre_local_counts - total_kept_count
RetainPermutations(G, keep)

remapped_configurations <- empty list
for each configuration C in configurations do
    append RemapConfiguration(C, remap) to remapped_configurations
end for

return (removed, remapped_configurations)
end procedure

```

Listing 13: Retain permutations and rebuild graph indexes

```

procedure RetainPermutations(graph G, keep)
    for each minigrid i do
        new_nodes[i] <- empty list
        new_payloads[i] <- empty list

        for each old permutation id p in keep[i] do
            node <- G.minigrids[i][p]
            payload <- G.payloads[i][node.payload_id]
            node.payload_id <- length(new_payloads[i])
            append payload to new_payloads[i]
            append node to new_nodes[i]
        end for
    end for

    G.minigrids <- new_nodes
    G.payloads <- new_payloads
    rebuild_row_and_column_signature_indexes
    BuildCompatibilityEdges(G)
end procedure

```

9 Configuration Search

This algorithm corresponds to `Search` in `src/solver/extraction.rs`. It is used both by exact pruning and by extraction when configurations are not already supplied.

Listing 14: Search all compatible minigrid configurations

```

procedure SearchAllConfigurations(graph G, mode)

```

```

for each minigrid i do
  if G has zero permutations for i then
    return empty list
  end if

  domains[i] <- all permutation ids for minigrid i
  assignments[i] <- None
end for

solutions <- empty list
trail <- empty stack
Backtrack(G, domains, assignments, trail, solutions, mode)
return solutions
end procedure

```

Listing 15: Backtracking with MRV and propagation

```

procedure Backtrack(graph G, domains, assignments, trail, solutions, mode)
  if mode has solution cap and length(solutions) >= cap then
    return
  end if

  if every minigrid is assigned then
    append assignments as complete configuration to solutions
    return
  end if

  i <- SelectNextMinigrid(domains, assignments)
  if i = None then
    return
  end if

  candidates <- all permutation ids in domains[i]
  for each permutation p in candidates do
    marker <- size(trail)

    if AssignAndPropagate(G, domains, assignments, trail, i, p) then
      Backtrack(G, domains, assignments, trail, solutions, mode)
    end if

    UndoTo(trail, domains, assignments, marker)
  end for
end procedure

function SelectNextMinigrid(domains, assignments)
  best <- None
  best_count <- infinity

  for each minigrid i do
    if assignments[i] is set then
      continue
    end if

    count <- size(domains[i])
    if count < best_count then
      best <- i
      best_count <- count
    end if
  end for
end function

```

```

        if best_count = 0 then
            break
        end if
    end for

    return best
end function

```

Listing 16: Assignment, propagation, and trail-based undo

```

procedure AssignAndPropagate(graph G, domains, assignments, trail, i, p)
    queue <- [(i, p)]
    SaveState(trail, domains, assignments, i)
    assignments[i] <- p
    domains[i] <- singleton set {p}

    while queue is not empty do
        (current_i, current_p) <- pop(queue)

        for each minigrid j do
            if RelationshipBetween(current_i, j) = NotRelated then
                continue
            end if

            compatible <- CompatibleSet(G, current_i, current_p, j)

            if assignments[j] = Some(q) then
                if q is not in compatible then
                    return false
                end if
                continue
            end if

            SaveState(trail, domains, assignments, j)
            changed <- Intersect(domains[j], compatible)

            if domains[j] is empty then
                return false
            end if

            if changed and size(domains[j]) = 1 then
                q <- only element of domains[j]
                assignments[j] <- q
                push (j, q) into queue
            end if
        end for
    end while

    return true
end procedure

procedure UndoTo(trail, domains, assignments, marker)
    while size(trail) > marker do
        entry <- pop(trail)
        domains[entry.minigrid] <- entry.domain
        assignments[entry.minigrid] <- entry.assignment
    end while

```

```
end procedure
```

10 Solution Extraction and Classification

This algorithm corresponds to `Extractor::run_with_configurations`, `Extractor::reconstruct`, and the classification logic in `src/solver/mod.rs`.

Listing 17: Extract and validate solved boards

```
procedure ExtractSolutions(graph G, configurations)
  solutions <- empty list

  for each configuration C in configurations do
    board <- ReconstructBoard(G, C)

    if IsValidSudokuBoard(board) then
      append Solution(board, C) to solutions
    end if
  end for

  return solutions
end procedure

procedure ReconstructBoard(graph G, configuration C)
  initialize cells[N][N] with zeros

  for each minigrid id i do
    p <- C[i]
    perm_cells <- G.permutation_cells(i, p)
    base_row <- floor(i / K) * K
    base_col <- (i mod K) * K

    for local index t in 0 .. N - 1 do
      local_row <- floor(t / K)
      local_col <- t mod K
      row <- base_row + local_row
      col <- base_col + local_col
      cells[row][col] <- perm_cells[t]
    end for
  end for

  return Board(cells)
end procedure

function Classify(solutions)
  if length(solutions) = 0 then
    return Unsolvable
  else if length(solutions) = 1 then
    return Unique
  else
    return Ambiguous(length(solutions))
  end if
end function
```

11 Algorithm Map to Source Files

- Complete pipeline: `src/solver/mod.rs`
- Mask initialization: `src/types/masks.rs`
- Hidden singles: `src/solver/heuristics.rs`
- Minigrid permutation generation: `src/solver/permutations.rs`
- Graph construction and compatibility: `src/types/graph/mod.rs`, `src/types/graph/node.rs`
- Local and exact pruning: `src/solver/pruning.rs`
- Configuration search and reconstruction: `src/solver/extraction.rs`